

Avaliação

- 1 - Conceitos fundamentais
- 2 - UML Diagrama de Classe
- 3 - Polimorfismo e o Padrão de Projeto Strategy
- ~~4 - Encapsulamento e o Padrão de Projeto Façade~~
- 5 - Tratamento de Exceções
- 6 - Coleções

Classe, atributo e método

- **Definições:** Classe é um modelo que define: - atributos e métodos
- **Atributos:** São as propriedades/características dos objetos.
- **Métodos:** São as funções ou procedimentos que descrevem os comportamentos ou ações que os objetos dessa classe podem realizar.

```
public class Pessoa {  
    private String nome;  
    private int idade;  
  
    public void apresentar() {  
        System.out.println("Sou " + nome + ", " + idade + " anos");  
    }  
  
    public String getNome() { return nome; }  
}
```

Objeto

- **Definições:** Objeto é uma instância de uma classe, uma entidade concreta criada a partir do modelo da classe.

Cada objeto possui seu próprio estado (valores dos atributos) e pode executar os comportamentos definidos na classe.

```
public class ExemploObjeto {  
    public static void main(String[] args) {  
        Pessoa p1 = new Pessoa("João", 25);  
        Pessoa p2 = new Pessoa("Maria", 30);  
  
        p1.apresentar();  
        p2.apresentar();  
  
        System.out.println(p1.getNome());  
        System.out.println(p2.getNome());  
    }  
}
```

Herança

- **Definições:** Herança é o mecanismo que permite criar uma nova classe (filha/subclasse) baseada em uma classe existente (pai/superclasse), herdando seus atributos e métodos, promovendo reutilização de código e hierarquias de classes.

```
public class PessoaJuridica extends Pessoa {  
    private int tempoAtividade;  
  
    public PessoaJuridica(String nome, int tempoAtividade) {  
        this.nome = nome;  
        this.tempoAtividade = tempoAtividade;  
    }  
  
    @Override  
    public void apresentar() {  
        System.out.println("Sou a empresa " + nome  
            + ", atuando há " + tempoAtividade + " anos.");  
    }  
}
```

Polimorfismo

Polimorfismo é a capacidade de objetos de diferentes classes responderem de forma específica a um método.

Permite que um mesmo código trabalhe com diferentes tipos de objetos de forma transparente.

```
public class ExemploPolimorfismo {  
    public static void main(String[] args) {  
        Pessoa p1 = new Pessoa("João", 25);  
        Pessoa p2 = new Pessoa("Maria", 30);  
        Pessoa p3 = new PessoaJuridica("ACME SA", 10);  
  
        p1.apresentar();  
        p2.apresentar();  
        p3.apresentar();  
  
        System.out.println(p1.getNome());  
        System.out.println(p2.getNome());  
    }  
}
```

Encapsulamento

Encapsulamento é o princípio de ocultar os detalhes internos de uma classe e expor apenas o que é necessário. Permite controlar o acesso aos dados e proteger a integridade do objeto. Exemplo: Os atributos são geralmente privados, e o acesso a eles é feito através de métodos públicos (getters e setters).

```
public class ContaCorrente {  
    private double saldo;  
    private String senha;  
  
    public ContaCorrente(String senha) {  
        this.senha = senha;  
        this.saldo = 0.0;  
    }  
  
    public double getSaldo(String senhaInformada) {  
        if (senha.equals(senhaInformada)) {  
            return saldo;  
        }  
        return -1; // senha incorreta  
    }  
  
    public void depositar(double valor) {  
        double valorFinal = this.saldo + valor;  
        this.saldo = valorFinal;  
    }  
}
```

UML - Diagrama de Classes

O diagrama de classes UML é usado para representar as classes de um sistema e seus relacionamentos.

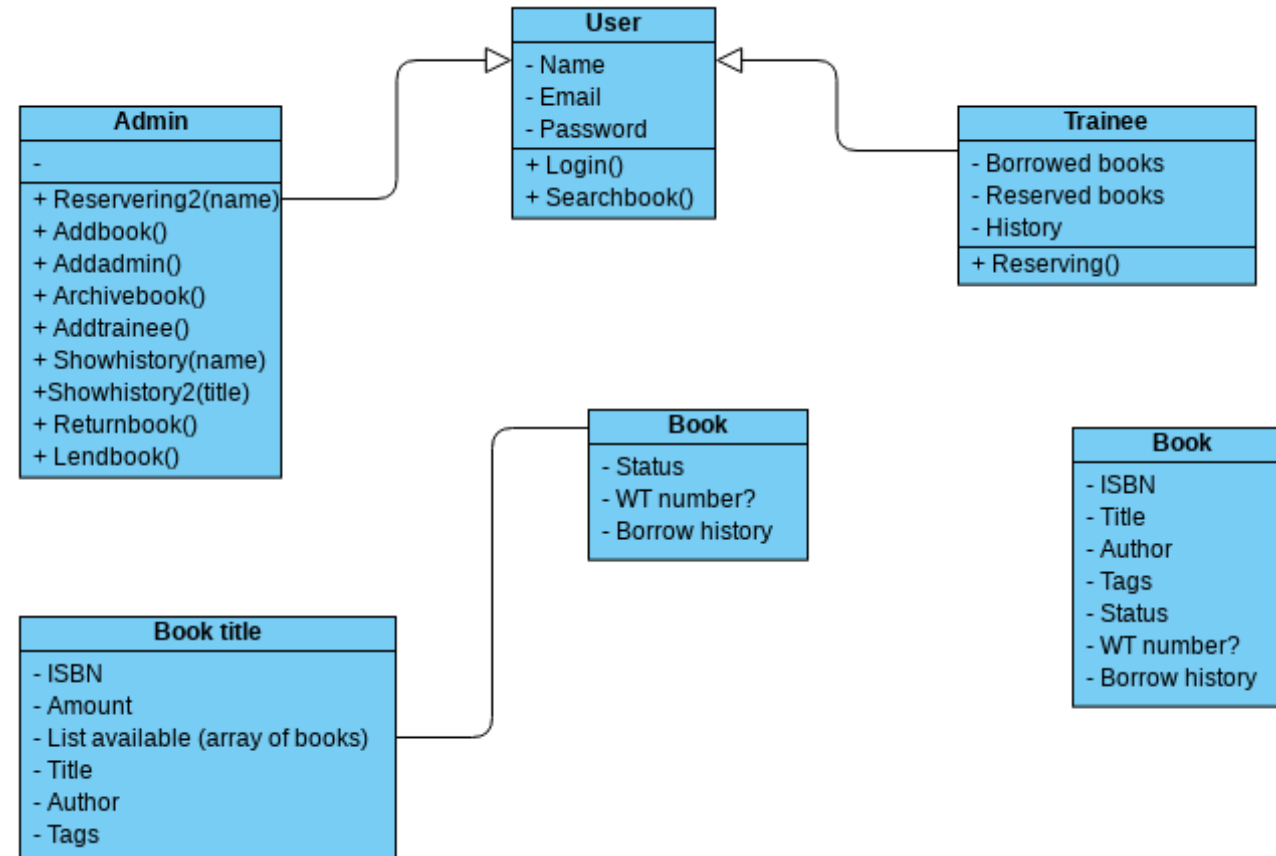
Classes: retângulos tripartidos

Atributos e métodos: listados dentro das classes

Relacionamentos: linhas conectando as classes

Modificadores de acesso:

- **+** público **-** privado



Polimorfismo e o Padrão de Projeto Strategy

Polimorfismo permite que diferentes classes implementem o mesmo método de maneiras distintas. O padrão Strategy utiliza esse conceito para definir uma família de algoritmos, encapsulando cada um e tornando-os intercambiáveis.

- Permite que o algoritmo varie independentemente dos clientes que o utilizam.
- Facilita a **extensibilidade** - novos algoritmos podem ser adicionados sem alterar o código existente.
- Baseado no princípio "**programar para interfaces, não para implementações**".

Polimorfismo e o Padrão de Projeto Strategy

```
// Interface Strategy
public interface CalculaValorPagamento {
    BigDecimal calcularValor(int horaInicial, int horaFinal,
                             BigDecimal valorHora);
}

// Implementação concreta do Strategy
public class PagamentoPorHoraNormal implements CalculaValorPagamento{
    @Override
    public BigDecimal calcularValor(int horaInicial, int horaFinal,
    BigDecimal valorHora) {
        //Considerar hora inicial 8 o início do expediente,
        // e hora final 18
        int horaInicialAux = Math.max(8, horaInicial);
        int horaFinalAux = Math.min(18, horaFinal);
        if (horaFinal <= horaInicial) {
            return BigDecimal.ZERO; // Não há horas válidas
        }
        return valorHora.multiply(new BigDecimal(horaFinalAux
        - horaInicialAux));
    }
}
```

```
public class Pagamento {
    // A classe Pagamento utiliza a estratégia sem conhecer os detalhes
    private CalculaValorPagamento estrategia;

    public Pagamento(CalculaValorPagamento estrategia) {
        this.estrategia = estrategia;
    }

    public void setEstrategia(CalculaValorPagamento estrategia) {
        this.estrategia = estrategia;
    }

    public BigDecimal calcularPagamento(int horaInicial, int horaFinal,
    BigDecimal valorHora) {
        return estrategia.calcularValor(horaInicial, horaFinal, valorHora);
    }
}
```

```
public static void main(String[] args) {
    Pagamento pagamento = new Pagamento(new PagamentoPorHoraNormal());
    BigDecimal valorHora = new BigDecimal("50.00");
    BigDecimal valor = pagamento.calcularPagamento(9, 17, valorHora);
    System.out.println("Valor do pagamento: R$ " + valor);
}
```

Encapsulamento e o Padrão de Projeto Façade

Encapsulamento oculta a complexidade de um sistema, expondo apenas uma interface simplificada. O padrão Façade aplica esse princípio ao fornecer uma interface unificada para um conjunto de interfaces em um subsistema, facilitando o uso do sistema.

```
public class SistemaPagamentoFacade {  
    private ProcessadorCartaoCredito processadorCartao;  
    private ServicoPagamentoOnline servicoPagamento;  
    private GerenciadorFaturas gerenciadorFaturas;  
  
    public SistemaPagamentoFacade() {  
        this.processadorCartao = new ProcessadorCartaoCredito();  
        this.servicoPagamento = new ServicoPagamentoOnline();  
        this.gerenciadorFaturas = new GerenciadorFaturas();  
    }  
    // Métodos simplificados para os clientes  
    public boolean processarPagamentoCartao(String numeroCartao,  
                                             double valor) {  
        if (processadorCartao.validarCartao(numeroCartao)) {  
            return processadorCartao.efetuarPagamento(numeroCartao,  
                                                         valor);  
        }  
        return false;  
    }  
}
```

Tratamento de Exceções em Orientação a Objetos

Tratamento de exceções é um mecanismo para lidar com erros ou condições excepcionais que ocorrem durante a execução de um programa. Em POO, isso é feito usando blocos try-catch-finally para capturar e tratar exceções, garantindo que o programa possa continuar a funcionar ao invés de falhar abruptamente.

Tipos de exceções

- **Checked Exceptions:** devem ser tratadas obrigatoriamente.
- **Unchecked Exceptions:** não precisam ser declaradas.

Tratamento de Exceções em Orientação a Objetos

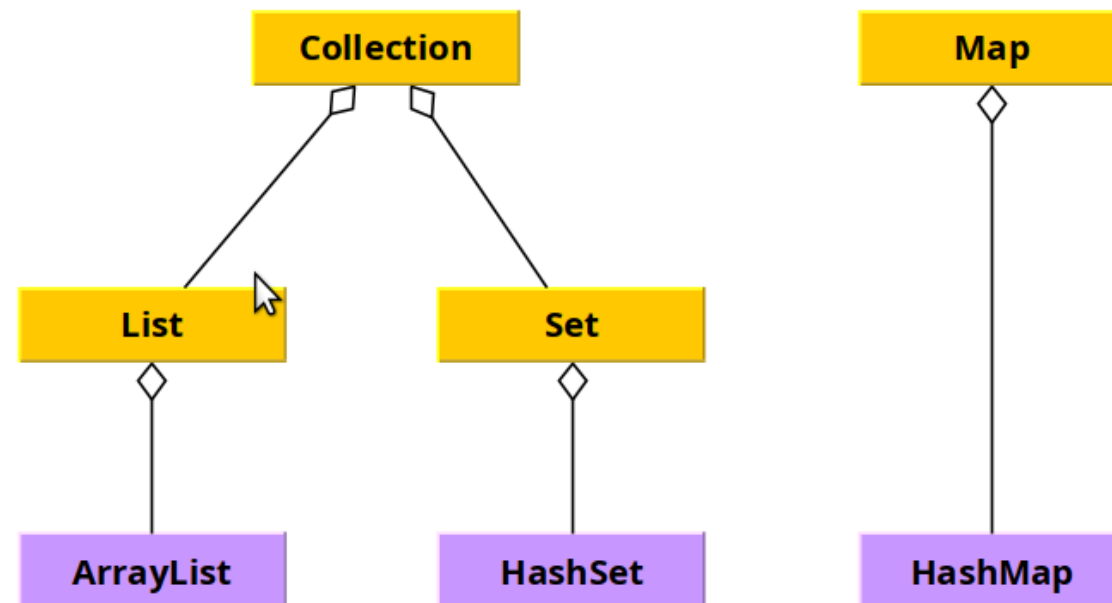
```
public class ContaCorrente {  
    public double getSaldo(String senhaInformada) {  
        if (senha.equals(senhaInformada)) {  
            return saldo;  
        }  
        //return -1; // senha incorreta  
        throw new IllegalArgumentException("Senha incorreta");  
    }  
}
```

```
public class ExemploExcecao {  
    public static void main(String[] args) {  
        ContaCorrente conta = new ContaCorrente("1234");  
        try {  
            double saldo = conta.getSaldo("0000");  
            System.out.println("Saldo: " + saldo);  
        } catch (Exception e) {  
            System.out.println("Erro: " + e.getMessage());  
        } finally {  
            System.out.println("Operação finalizada.");  
        }  
    }  
}
```

Coleções em Orientação a Objetos

Coleções são estruturas de dados que armazenam múltiplos elementos, as coleções agrupam objetos de forma

- Adicionar
- Remover
- Tamanho
- Iterar



```

import java.util.*;

public class ExemploSet {
    public static void main(String[] args) {

        // Operações de conjunto
        Set<String> set1 = Set.of("A", "B", "C");
        Set<String> set2 = Set.of("B", "C", "D");

        Set<String> uniao = new HashSet<>(set1);
        uniao.addAll(set2); // União

        Set<String> intersecao = new HashSet<>(set1);
        intersecao.retainAll(set2); // Interseção

        System.out.println("União: " + uniao);
        System.out.println("Interseção: " + intersecao);

    }
}

```

```

import java.util.*;

public class ExemploList {
    public static void main(String[] args) {
        List<String> nomes = new ArrayList<>();

        // Adicionando elementos
        nomes.add("João");
        nomes.add("Maria");
        nomes.add("Pedro");
        nomes.add("Maria"); // Permite duplicatas

        // Operações principais
        System.out.println("Primeiro: " + nomes.get(0));
        System.out.println("Tamanho: " + nomes.size());

        nomes.set(1, "Ana"); // Substituir
        nomes.remove("Pedro"); // Remover por valor
        nomes.remove(0); // Remover por índice

        // Busca
        boolean tem = nomes.contains("Maria");
        int indice = nomes.indexOf("Maria");

        System.out.println("Contém Maria: " + tem);
        System.out.println("Índice: " + indice);

    }
}

```

Exercício aula 5

- Implementar uma ordenação natural para a classe Produto (pelo nome do produto).
- Implementar uma ordenação personalizada para a classe Produto (pela quantidade em estoque).
- Criar uma lista de produtos, adicionar alguns produtos e ordenar usando ambas as ordenações.

